



PROCEDIMIENTOS ALMACENADOS Y FUNCIONES

Adrián Alonso Ridao



2º ASIR
2ª EVALUACIÓN
ADMINISTRACIÓN DE BASE DE DATOS

Contenido

Procedimientos Almacenados 2

 Procedimiento 1 2

 Procedimiento 2 3

 Procedimiento 3 4

 Procedimiento 4 4

 Procedimiento 5 5

Funciones 6

 Función 1 6

 Función 2 7

 Función 3 8

 Función 4 9

 Función 5 11

Procedimientos Almacenados

Procedimiento 1

Crea un procedimiento llamado ListarClientesPorCiudad que reciba como parámetro el nombre de una ciudad y muestre todos los clientes que viven en esa ciudad.

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE ListarClientesPorCiudad(IN nombreCiudad VARCHAR(50))
```

```
    -> BEGIN
```

```
    -> SELECT * FROM cliente
```

```
    -> WHERE ciudad=nombreCiudad;
```

```
    -> END //
```

```
Query OK, 0 rows affected (0.01 sec)
```



```
mysql> DELIMITER ;
```

```
mysql> CALL ListarClientesPorCiudad('Sevilla');
```

id	nombre	apellido1	apellido2	ciudad	categor?a	total_compras	estado_credito
3	Adolfo	Rubio	Flores	Sevilla	NULL	0	Pendiente
7	Pilar	Ruiz	NULL	Sevilla	300	0	Pendiente
10	Daniel	Santana	Loyola	Sevilla	125	0	Pendiente

```
3 rows in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.02 sec)
```

Procedimiento 2

Crea un procedimiento llamado `MostraPedidosRecientes` que reciba un número `N` como parámetro y muestre los `N` pedidos más recientes ordenados por fecha descendente.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE MostrarPedidosRecientes(IN numero INT)
  -> BEGIN
  -> SELECT * FROM pedido
  -> ORDER BY fecha DESC
  -> LIMIT numero;
  -> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
mysql> CALL MostrarPedidosRecientes(5);
```

id	total	fecha	id_cliente	id_comercial
17	500	2024-01-09	2	1
15	370.85	2019-03-11	1	5
16	2389.23	2019-03-11	1	5
13	545.75	2019-01-25	6	1
8	1983.43	2017-10-10	4	6

```
5 rows in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.02 sec)
```

Procedimiento 3

Crea un procedimiento llamado **ActualizarTotalComprasCliente** que reciba el ID de un cliente y recalculé su total de compras sumando todos sus pedidos.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE ActualizarTotalComprasCliente(IN numero_cliente INT)
  -> BEGIN
  -> SELECT COUNT(*) AS TotalPedidos
  -> FROM pedido
  -> WHERE id_cliente = numero_cliente;
  -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL ActualizarTotalComprasCliente(1);
+-----+
| TotalPedidos |
+-----+
|          3 |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

Procedimiento 4

Crea un procedimiento llamado **ListarComercialesConComision** que muestre todos los comerciales con su comisión, pero solo aquellos cuya comisión sea mayor que un valor mínimo pasado como parámetro.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE ListarComercialesConComision(IN numero_comision FLOAT)
  -> BEGIN
  -> SELECT * FROM comercial
  -> WHERE comisión > numero_comision;
  -> END //
Query OK, 0 rows affected, 2 warnings (0.01 sec)

mysql> DELIMITER ;
mysql> CALL ListarComercialesConComision(0.12);
+-----+-----+-----+-----+-----+
| id | nombre | apellido1 | apellido2 | comisi?n |
+-----+-----+-----+-----+-----+
| 1 | Daniel | S | Vega | 0.17 |
| 2 | Juan | G | L | 0.13 |
| 4 | Marta | Herrera | Gil | 0.14 |
| 6 | Manuel | Dom | Hern | 0.13 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

Query OK, 0 rows affected, 2 warnings (0.01 sec)
```

Procedimiento 5

Crea un procedimiento llamado `InsertarNuevoComercial` que reciba nombre, apellido1, apellido2 y comisión, e inserte un nuevo comercial en la tabla.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE InsertarNuevoComercial(
  ->     IN n_nombre VARCHAR(50),
  ->     IN n_apellido1 VARCHAR(50),
  ->     IN n_apellido2 VARCHAR(50),
  ->     IN n_comision FLOAT
  -> )
  -> BEGIN
  -> INSERT INTO comercial (nombre, apellido1, apellido2, comisión)
  -> VALUES (n_nombre, n_apellido1, n_apellido2, n_comision);
  -> END //
Query OK, 0 rows affected, 2 warnings (0.01 sec)

mysql> DELIMITER ;
mysql> CALL InsertarNuevoComercial('Juan', 'Pérez', 'García', 0.15);
Query OK, 1 row affected, 2 warnings (0.01 sec)

mysql> SELECT * FROM comercial;
+----+-----+-----+-----+-----+
| id | nombre | apellido1 | apellido2 | comisi?n |
+----+-----+-----+-----+-----+
| 1 | Daniel | S | Vega | 0.17 |
| 2 | Juan | G | L | 0.13 |
| 3 | Diego | Flores | Salas | 0.11 |
| 4 | Marta | Herrera | Gil | 0.14 |
| 5 | Antonio | Carretero | Ortega | 0.12 |
| 6 | Manuel | Dom | Hern | 0.13 |
| 7 | Antonio | Vega | Hern | 0.11 |
| 8 | Alfredo | Ruiz | Flores | 0.07 |
| 9 | Juan | P?rez | Garc?a | 0.15 |
+----+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql>
```

Funciones

Función 1

Crea una función llamada `TotalComprasCliente` que reciba el ID de un cliente y devuelva la suma total de todos sus pedidos.

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION TotalComprasCliente(n_cliente INT)
```

```
    -> RETURNS INT
```

```
    -> READS SQL DATA
```

```
    -> BEGIN
```

```
    -> DECLARE total_pedidos INT;
```

```
    -> SELECT COUNT(*) INTO total_pedidos FROM pedido
```

```
    -> WHERE id_cliente = n_cliente;
```

```
    -> RETURN total_pedidos;
```

```
    -> END //
```

```
Query OK, 0 rows affected (0.01 sec)
```



```
mysql> DELIMITER ;
```

```
mysql> SELECT TotalComprasCliente(1);
```

TotalComprasCliente(1)
3

```
1 row in set (0.01 sec)
```



```
mysql> |
```

Función 2

Crea una función llamada **NombreCompletoCliente** que reciba el ID de un cliente y devuelva su nombre completo en formato “Nombre Apellido1 Apellido2”.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION NombreCompletoCliente(n_id INT)
  -> RETURNS VARCHAR(100)
  -> READS SQL DATA
  -> BEGIN
  -> DECLARE nombre_completo VARCHAR(100);
  -> SELECT CONCAT(nombre, ' ', apellido1, ' ', apellido2) INTO nombre_completo FROM cliente
  -> WHERE id = n_id;
  -> RETURN nombre_completo;
  -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> SELECT NombreCompletoCliente(1);
+-----+
| NombreCompletoCliente(1) |
+-----+
| Aar Rivero G             |
+-----+
1 row in set (0.00 sec)
```


Función 3

Crea una función llamada `DiasDesdeUltimoPedido` que reciba el ID de un cliente y devuelva cuántos días han pasado desde su último pedido hasta hoy.

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION DiasDesdeUltimoPedido(n_id INT)
```

```
  -> RETURNS INT
```

```
  -> READS SQL DATA
```

```
  -> BEGIN
```

```
  -> DECLARE dias_pasados INT;
```

```
  -> DECLARE ultima_fecha DATE;
```

```
  -> SELECT MAX(fecha) INTO ultima_fecha FROM pedido
```

```
  -> WHERE id_cliente = n_id;
```

```
  -> IF ultima_fecha IS NULL THEN
```

```
  -> RETURN NULL;
```

```
  -> ELSE
```

```
  -> SET dias_pasados = DATEDIFF(NOW(), ultima_fecha);
```

```
  -> RETURN dias_pasados;
```

```
  -> END IF;
```

```
  -> END //
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> DELIMITER ;
```

```
mysql> SELECT DiasDesdeUltimoPedido(2);
```

DiasDesdeUltimoPedido(2)
705

1 row in set (0.01 sec)

Función 4

Crea una función llamada `CategoriaClienteTexto` que reciba el número de categoría y devuelva:

- “ALTA” si categoría ≥ 200
- “MEDIA” si categoría ≥ 100
- “BAJA” para el resto

```
mysql> DELIMITER //
mysql> CREATE FUNCTION CategoriaClienteTexto(n_categoria INT)
  -> RETURNS VARCHAR(10)
  -> READS SQL DATA
  -> BEGIN
  -> IF n_categoria >= 200 THEN
  -> RETURN 'ALTA';
  -> ELSEIF n_categoria >= 100 THEN
  -> RETURN 'MEDIA';
  -> ELSE
  -> RETURN 'BAJA';
  -> END IF;
  -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
```

```
mysql> SELECT CategoriaClienteTexto(300);
```

CategoriaClienteTexto(300)
ALTA

```
1 row in set (0.00 sec)
```

```
mysql> SELECT CategoriaClienteTexto(150);
```

CategoriaClienteTexto(150)
MEDIA

```
1 row in set (0.00 sec)
```

```
mysql> SELECT CategoriaClienteTexto(50);
```

CategoriaClienteTexto(50)
BAJA

```
1 row in set (0.00 sec)
```

Función 5

Crea una función llamada **ComisionEnPorcentaje** que reciba el ID de un comercial y devuelva su comisión en formato porcentaje (ej: "15.50%").

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION ComisionEnPorcentaje(n_id INT)
-> RETURNS VARCHAR(20)
-> READS SQL DATA
-> BEGIN
-> DECLARE resultado VARCHAR(20);
-> DECLARE valor_real FLOAT;
-> SELECT comisión INTO valor_real FROM comercial
-> WHERE id = n_id;
-> IF valor_real IS NULL THEN
-> RETURN NULL;
-> ELSE
-> SET resultado = CONCAT(FORMAT(valor_real * 100, 2), '%');
-> RETURN resultado;
-> END IF;
-> END //
```

Query OK, 0 rows affected, 2 warnings (0.01 sec)

```
mysql> DELIMITER ;
```

```
mysql> SELECT ComisionEnPorcentaje(1);
```

ComisionEnPorcentaje(1)
17.00%

1 row in set, 2 warnings (0.01 sec)